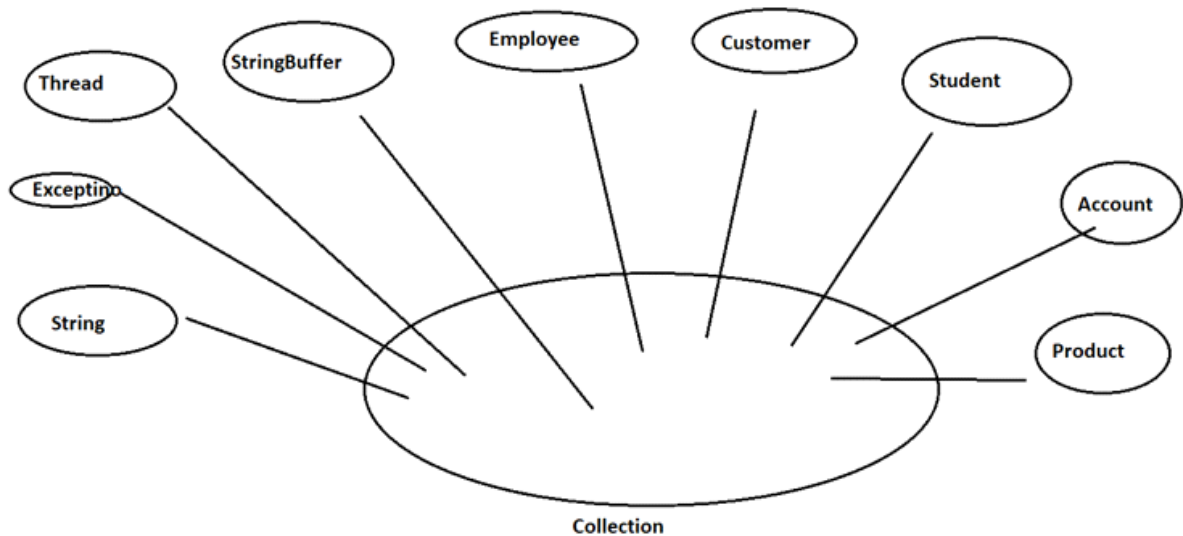


Collection is an object, it able to represent a group of other objects.



Q) In Java Applications, To Represent A Group Of Other Elements We Have Already Arrays Then What Is The Requirement To Use Collections?

OR

Q) What Are The Differences Between Array And Collection?

- Arrays are having fixed size in nature. In case of arrays, we are able to add the elements up to the specified size only, we are unable to add the elements over its size, if we are trying to add elements over its size then JVM will rise an exception like "java.lang.ArrayIndexOutOfBoundsException".

EX:

```
Student[] std=new Student[3];  
std[0]=new Student();  
std[1]=new Student();  
std[2]=new Student();  
std[3]=new Student();--> ArrayIndexOutOfBoundsException
```

Collections are having dynamically growable nature, even if we add the elements over its size then JVM will not rise any exception.

EX:

```
ArrayList al=new ArrayList(3);  
al.add(new Student());  
al.add(new Student());  
al.add(new Student());  
al.add(new Student());--> No Exception
```

- In Java, by default, Arrays are able to allow homogeneous elements, if we are trying to add the elements which are not same Array data type then Compiler will rise an error like "Incompatible Types".

EX:

```
Student[] std=new Student[3];  
std[0]=new Student();  
std[1]=new Student();  
std[2]=new Customer();--> Incompatible Types Error
```

In Java, by default, Collections are able to allow heterogeneous elements, even we add different types of elements Compiler will not rise any error.

EX:

```
ArrayList al=new ArrayList(3);  
al.add(new Student());  
al.add(new Employee());---> No Error  
al.add(new Customer());----> No Error
```

- Arrays are not having predefined methods to perform searching and sorting operations over the elements, in case of arrays to perform searching and sorting operations developers have to provide their own logic.

In case of Collections, predefined methods or predefined Collections are defined to perform Searching and sorting operations over the elements.

EX: In Collections, TreeSet was provided to perform sorting order.

```
TreeSet ts=new TreeSet();  
ts.add("B");  
ts.add("E");  
ts.add("A");  
ts.add("D");  
ts.add("C");  
ts.add("F");  
System.out.println(ts);
```

OUTPUT: [A,B,C,D,E,F]

- Arrays are able to allow only one type of elements, so Arrays are able to improve Typedness in java applications and they are able to perform Type safe operations.

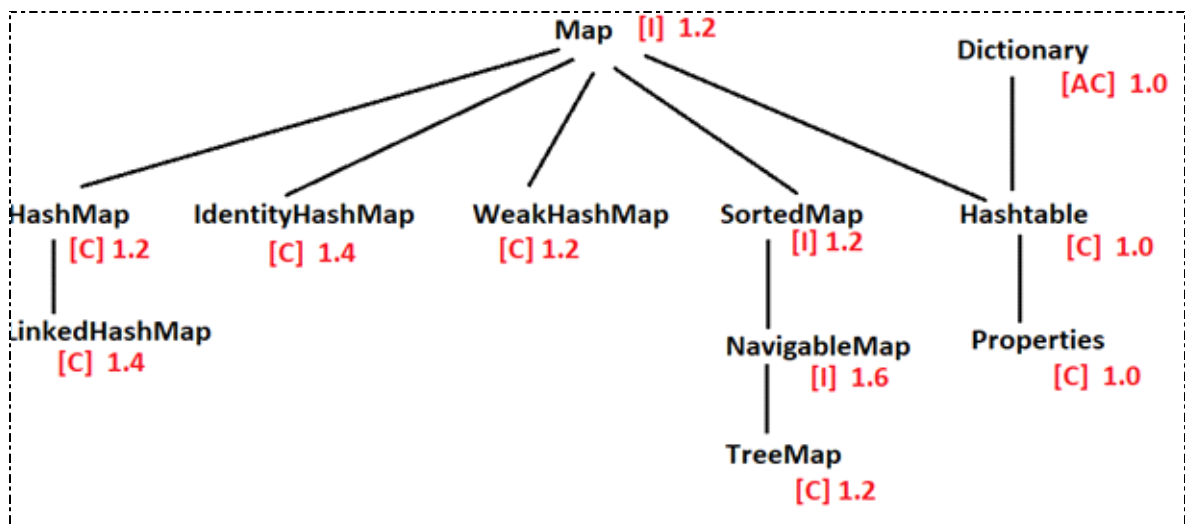
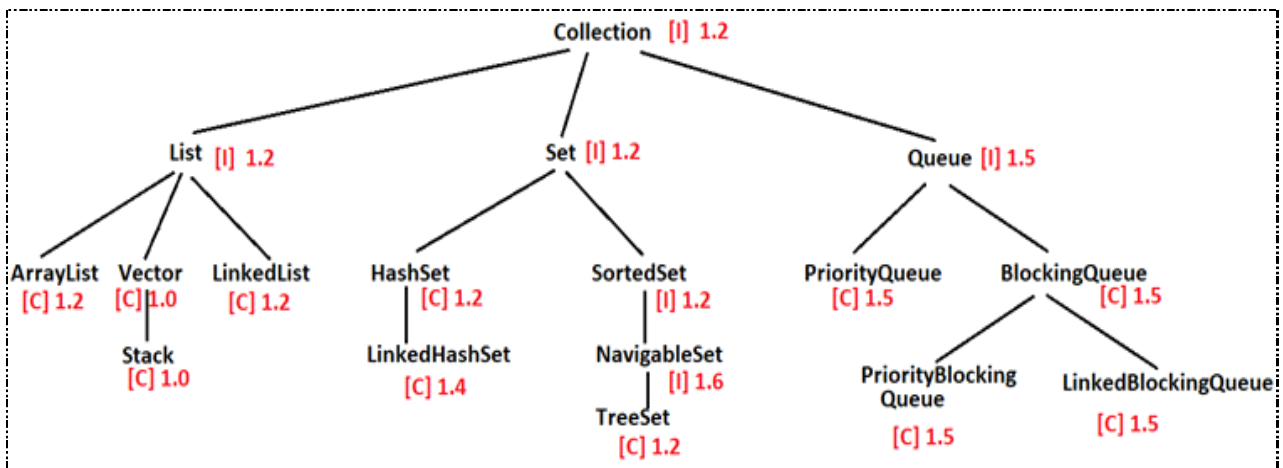
Collections are able to allow different types of elements, so Collections are able to reduce typedness in java applications and they are unable to perform Type safe operations.

- If we know the no of elements in advance at the time of writing java applications then Arrays are better to use in java applications and they will provide very good performance in java applications, but, But Arrays are not flexible to design applications.

In java applications, Collections are able to provide less performance, but, they will provide flexibility to design applications.

To represent Collection objects in java applications, JAVA has provided predefined classes and interfaces in the form of java.util package called as "Collection Framework".

Q) What are the classes and interfaces are existed in java.util package to represent Collections?



Q) What are the differences between Collection and Map?

Collections are able to store all the elements individually, not in the form of Key-value pairs.

EX: To store 10 Employee objects we will use Collection.

Maps are able to store all the elements in the form of Key-value pairs.

EX: To represent Telephone Directory, where we are representing phone number and Customer Name we have to use Maps.

Q) What are the differences between List and Set?

- List is index based, it able to allow all the elements as per indexing.
Set is not index based, it able to allow all the elements on the basis of elements hash code values.
- List is able to allow duplicate elements.
Set is not allowing duplicate elements.
- List is able to allow any number of null values.
Set is able to allow only one null value.
- List is following insertion order.
Set is not following insertion order by default.

Note: LinkedHashSet is following insertion order.

- List is not following sorting order.
Sets are not following sorting order by default.

Note: SortedSet, NavigableSet and TreeSet are following Sorting order.

- List is able to allow heterogeneous elements.
Sets are able to allow heterogeneous elements by default.

Note: SortedSet, NavigableSet and TreeSet are allowing only Homogeneous elements.

Collection:

- It is an interface provided by JAVA along with JDK 1.2 version.
- It able to represent a group of individual elements as single unit.
- It has provided the following methods common to every implementation class.

- public boolean add(Object obj)

This method is able to add the specified element to Collection object. If the specified element is added successfully then add(-) method will return "true" value. If the specified element is not added successfully then add() method will return "false" value.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         HashSet hs=new HashSet();
7)         System.out.println(hs.add("A"));
8)         hs.add("B");
9)         hs.add("C");
10)        hs.add("D");
11)        System.out.println(hs);
12)        System.out.println(hs.add("A"));
13)        System.out.println(hs);
14)    }
15)}
```

OUTPUT:

true

[A,B,C,D]

false

[A,B,C,D]

- **public boolean addAll(Collection c)**

This method can be used to add all the elements of the specified Collection to the present Collection object. If addition operation is success then addAll(-) method will return "true" value, if addition operation is failure then addAll() method will return "false" value.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         HashSet hs=new HashSet();
7)         hs.add("A");
8)         hs.add("B");
9)         hs.add("C");
10)        hs.add("D");
11)        System.out.println(hs);
12)        HashSet hs1=new HashSet();
13)        System.out.println(hs1.addAll(hs));
14)        System.out.println(hs1);
15)        System.out.println(hs1.addAll(hs));
```

```
16)    System.out.println(hs1);
17)    }
18) }
```

OUTPUT:

```
[D,A,B,C]
true
[D,A,B,C]
false
[D,A,B,C]
```

- **public boolean remove(Object obj)**

This method can be used to remove the specified element from the Collection object. If remove operation is success then remove() method will return true value, if remove operation is failure then remove() method will return false value.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        System.out.println(al);
12)        System.out.println(al.remove("B"));
13)        System.out.println(al);
14)        System.out.println(al.remove("B"));
15)        System.out.println(al);
16)    }
17) }
```

OUTPUT:

```
[A,B,C,D]
true
[A,C,D]
false
[A,C,D]
```

- **public boolean removeAll(Collection c)**

This method can be used to remove all the elements of the specified Collection from the present Collection object. If remove operation is success then removeAll() method will return true value. If remove operation is not success then removeAll() method will return false value.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        al.add("E");
12)        al.add("F");
13)        System.out.println(al);
14)        ArrayList al1=new ArrayList();
15)        al1.add("B");
16)        al1.add("C");
17)        al1.add("D");
18)        System.out.println(al1);
19)        System.out.println(al.removeAll(al1));
20)        System.out.println(al);
21)        System.out.println(al.removeAll(al1));
22)        System.out.println(al);
23)    }
24) }
```

OUTPUT:

[A,B,C,D,E,F]

[B,C,D]

true

[A,E,F]

false

[A,E,F]

- **public boolean contains(Object obj)**

This method will check whether the specified element is existed or not in the Collection object. If the specified element is existed then this method will return "true" value. If the specified element is not existed then this method will return "false" value.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        al.add("E");
12)        al.add("F");
13)        System.out.println(al);
14)        System.out.println(al.contains("B"));
15)        System.out.println(al.contains("X"));
16)    }
17)}
```

OUTPUT:

[A,B,C,D,E,F]
true
false

- **public boolean containsAll(Collection c)**

This method will check whether all the elements of the specified Collection are available or not in the present Collection object. If all the elements are existed then containsAll() method will return true value, if atleast one element is not existed then containsAll() method will return false value.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
```



```

8)    al.add("B");
9)    al.add("C");
10)   al.add("D");
11)   al.add("E");
12)   al.add("F");
13)   System.out.println(al);
14)   ArrayList al1=new ArrayList();
15)   al1.add("B");
16)   al1.add("C");
17)   al1.add("D");
18)   System.out.println(al.containsAll(al1));
19)   al1.add("X");
20)   al1.add("Y");
21)   System.out.println(al.containsAll(al1));
22) }
23) }

```

OUTPUT:

[A,B,C,D,E,F]

true

false

- **public boolean retainAll(Collection c)**

This method will remove all the elements from the present Collection object except the elements which are existed in the specified Collection object. If at least one element is removed then retainAll() method will return true value. If no elements are removed then retainsAll() method will return false value.

EX:

```

1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        al.add("E");
12)        al.add("F");
13)        System.out.println(al);
14)        ArrayList al1=new ArrayList();
15)        al1.add("B");
16)        al1.add("C");

```

```

17)    al1.add("D");
18)    System.out.println(al1);
19)    System.out.println(al.retainAll(al1));
20)    System.out.println(al);
21)    System.out.println(al.retainAll(al1));
22)    System.out.println(al);
23) }
24) }

```

OUTPUT:

[A,B,C,D,E,F]

[B,C,D]

true

[B,C,D]

false

[B,C,D]

- **public int size()**

This method can be used to return an integer value representing the no of elements which are existed in the Collection object.

- **public void clear()**

This method can be used to remove all elements from Collection object.

- **public boolean isEmpty()**

This method can be used to check whether Collection object is empty or not. If the Collection object is empty then isEmpty() method will return "true" value. If the Collection object is not empty then isEmpty() method will return "false" value.

- **public Object[] toArray()**

This method will return all the elements of the Collection object in the form of Object[].

EX:

```

1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");

```

```

10)    al.add("D");
11)    al.add("E");
12)    al.add("F");
13)    System.out.println(al);
14)    System.out.println(al.size());
15)    Object[] obj=al.toArray();
16)    for(Object o: obj)
17)    {
18)        System.out.print(o+" ");
19)    }
20)    System.out.println();
21)    System.out.println(al.isEmpty());
22)    al.clear();
23)    System.out.println(al.isEmpty());
24)    System.out.println(al);
25) }
26) }

```

OUTPUT:

[A,B,C,D,E,F]

6

A B C D E F

false

true

[]

List:

- List is a direct child interface to Collection interface
- List was provided by JAVA along with its JDK1.2 version
- List is index based, it able to arrange all the elements as per indexing.
- List is able to allow duplicate elements.
- List is following insertion order.
- List is not following Sorting order.
- List is able to allow any number of null values.
- List is able to allow heterogeneous elements.

List interface has provided the following methods common to all of its implementation classes.

- **public void add(int index, Object obj)**
It able to add the specified element at the specified index value.
- **public Object set(int index, Object obj)**
It able to set the specified element at the specified index value.

Q) What is the difference between *add(--)* Method and *set(--)* Method?

- **add(--)** method is able to perform insert operation. If any element is existed at the specified element then add() method will insert the specified new element at the specified index value and add() method will adjust the existed element to next index value. If no element is existed at the specified index then add() method add the specified element at the specified index.
- **set(--)** method is able to perform replace operation. If any element is existed at the specified index then set() method will remove the existed element and set(-) method will add the specified element to the specified index and set() method will return the removed element. If no element is existed at the specified index value then set() method will rise an exception like java.lang.IndexOutOfBoundsException.
- **public Object get(int index)**
It will return an element available at the specified index value.
- **public Object remove(int index)**
It will remove and return an element available at the specified index value.
- **public int indexOf(Object obj)**
It will return an index value where the first occurrence of the specified element.
- **public int lastIndexOf(Object obj)**
It will return an index value where the last occurrence of the specified element.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        al.add("E");
12)        System.out.println(al);
13)        al.add(1,"X");
14)        System.out.println(al);
15)        al.add(6,"F");
```

```

16) System.out.println(al);
17) al.set(3,"Y");
18) System.out.println(al);
19) //al.set(7,"Z");--->IndexOutOfBoundsException
20) System.out.println(al.get(4));
21) System.out.println(al.remove(6));
22) System.out.println(al);
23) al.add(6,"X");
24) al.add(7,"B");
25) al.add(8,"X");
26) System.out.println(al);
27) System.out.println(al.indexOf("X"));
28) System.out.println(al.lastIndexOf("X"));
29) }
30) }

```

ArrayList:

- It was provided by JAVA along with JDK 1.2 version.
- It is a direct implementation class to List interface.
- It is index based.
- It allows duplicate elements.
- It follows insertion order.
- It will not follow sorting order.
- It allows heterogeneous elements.
- It allows any number of null values.
- Its internal data structure is "Resizable Array".
- Its initial capacity is 10 elements.
- Its incremental capacity ration is

$$\text{new_Capacity} = (\text{Current_Capacity} * 3/2) + 1$$
- It is best option for frequent retrieval operations.
- It is not synchronized.
- No method is synchronized method in ArrayList.
- It allows more than one thread to access data.
- It follows parallel execution.
- It will reduce execution time.
- It will improve application performance.
- It will not give guarantee for data consistency.
- It is not thread safe.
- It is not Legacy Collection.

Constructors:

- **public ArrayList()**

It can be used to create an empty ArrayList object with 10 elements as default capacity value.

EX: ArrayList al = new ArrayList();

- **public ArrayList(int capacity)**

It can be used to create an empty ArrayList object with the specified capacity.

EX: ArrayList al = new ArrayList(20);

- **public ArrayList(Collection c)**

It can be used to create an ArrayList object with all the elements of the specified Collection object.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al1=new ArrayList();
7)         al1.add("AAA");
8)         al1.add("BBB");
9)         al1.add("CCC");
10)        al1.add("DDD");
11)        System.out.println(al1);
12)        ArrayList al2=new ArrayList(al1);
13)        System.out.println(al2);
14)    }
15) }
```

OUTPUT:

[AAA,BBB,CCC,DDD]

[AAA,BBB,CCC,DDD]

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        al.add("E");
12)        System.out.println(al);
13)        al.add("B");
14)        System.out.println(al);
15)        al.add(new Integer(10));
16)        System.out.println(al);
17)        al.add(null);
18)        al.add(null);
19)        System.out.println(al);
20)    }
21)}
```

Vector:

- It was introduced in JDK1.0 version.
- It is Legacy Collection.
- It is a direct implementation class to List interface.
- It is index based.
- It allows duplicate elements.
- It follows insertion order.
- It will not follow sorting order.
- It allows heterogeneous elements.
- It allows any number of null values.
- Its internal data structure is "Resizable Array".
- Its initial capacity is 10 elements.
- It is best choice for frequent retrieval operations.
- It is not good for frequent insertions and deletion operations.
- Its incremental capacity is double the current capacity.
 $\text{New_capacity} = 2 * \text{Current_Capacity}$
- It is synchronized element.
- All the methods of vector class are synchronized.
- It allows only one thread at a time.
- It follows sequential execution.
- It will increase execution time.
- It will reduce application performance.

- It is giving guarantee for data consistency.
- It is threadsafe.

Constructors:

- **public Vector()**

It can be used to create an empty Vector object with the initial capacity 10 elements.

EX: Vector v = new Vector();
System.out.println(v.capacity());
OUTPUT: 10

- **public Vector(int capacity)**

It can be used to create an empty vector object with the specified capacity value.

EX: Vector v = new Vector(20);
System.out.println(v.capacity());
OUTPUT: 20

- **public Vector(int capacity, int incremental_Ratio)**

This constructor can be used to create an empty Vector object with the specified initial capacity and with the specified incremental ratio.

EX:

```

1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         Vector v=new Vector(5,5);
7)         System.out.println(v.capacity());
8)         for(int i=1;i<=6;i++)
9)         {
10)            v.add(i);
11)        }
12)        System.out.println(v.capacity());
13)        for(int i=7;i<=11;i++)
14)        {
15)            v.add(i);
16)        }
17)        System.out.println(v.capacity());
18)    }
19) }
```


OUTPUT:

5
10
15

- **public Vector(Collection c)**

This constructor can be used to create Vector object with all the elements of the specified Collection object.

EX:

```
1) import java.util.*;  
2) class Test  
3) {  
4)     public static void main(String[] args)  
5)     {  
6)         Vector v=new Vector();  
7)         v.add("A");  
8)         v.add("B");  
9)         v.add("C");  
10)        v.add("D");  
11)        System.out.println(v);  
12)        Vector v1=new Vector(v);  
13)        System.out.println(v1);  
14)    }  
15)}
```

OUTPUT:

[A,B,C,D]
[A,B,C,D]

Methods:

- **public void addElement(Object obj)**

It will add the specified element to Vector.

- **public Object firstElement()**

It will return first element of the Vector.

- **public Object lastElement()**

It will return last element of the Vector.

- **public Object elementAt(int index)**
It will return an element available at the specified index.
- **public void removeElement(Object obj)**
It will remove the specified element from Vector.
- **public void removeElementAt(int index)**
It will remove an element existed at the specified index value.
- **public void removeAllElements()**
It will remove all elements from Vector.

EX:

```

1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         Vector v=new Vector();
7)         v.addElement("A");
8)         v.addElement("B");
9)         v.addElement("C");
10)        v.addElement("D");
11)        v.addElement("E");
12)        System.out.println(v);
13)        System.out.println(v.firstElement());
14)        System.out.println(v.lastElement());
15)        System.out.println(v.elementAt(3));
16)        v.removeElement("D");
17)        System.out.println(v);
18)        v.removeElementAt(2);
19)        System.out.println(v);
20)        v.removeAllElements();
21)        System.out.println(v);
22)    }
23) }
```

Q) What are the differences between ArrayList and Vector?

- **ArrayList Class was introduced in JDK 1.2 Version**
Vector class was introduced in JDK1.0 version.
- **ArrayList is not Legacy Collection**
Vector is Legacy Collection.
- **ArrayList is not synchronized**
Vector is synchronized.
- **No method is synchronized Method in ArrayList**
Almost all the methods are synchronized methods in vector.
- **ArrayList allows more than one Thread at a Time to access Data**
Vector allows only one thread at a time to access data.
- **ArrayList follows parallel Execution**
Vector follows sequential execution.
- **ArrayList is able to reduce Application Execution Time**
Vector is able to increase application execution time.
- **ArrayList is able to improve Application Performance**
Vector is able to reduce application performance.
- **ArrayList is not giving guarantee for data consistency.**
Vector is giving guarantee for data consistency.
- **ArrayList is not threadsafe.**
Vector is threadsafe.
- **ArrayList Incremental Capacity is $(\text{Current Capacity} * 3/2) + 1$**
Vector incremental capacity is $2 * \text{Current_Capacity}$
- **We are unable to get Capacity Value of ArrayList, because, no capacity() Method in ArrayList Class.**
We can get capacity value of Vector, because, capacity() method is existed in vector class.

Stack:

It was introduced in JDK 1.0 version, it is a Legacy Collection and it is a child class to Vector class. It able to arrange all the elements as per "Last In First Out" [LIFO] alg.

Constructor:

`public Stack()`

It will create an empty Stack object.

EX: `Stack s = new Stack();`

Methods:

- `public void push(Object obj)`

It will add the specified element to Stack.

- `public Object pop()`

It will remove and return top of the stack.

- `public Object peek()`

It will return top of the stack.

- `public int search(Object obj)`

It will check whether the specified element is existed or not in the Stack, if the specified element is not existed then it will return '-1' value, if the specified element is existed then it will return its position.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         Stack s=new Stack();
7)         s.push("A");
8)         s.push("B");
9)         s.push("C");
10)        s.push("D");
11)        s.push("E");
12)        System.out.println(s);
13)        System.out.println(s.pop());
14)        System.out.println(s);
15)        System.out.println(s.peek());
```

```
16) System.out.println(s);
17) System.out.println(s.search("B"));
18) System.out.println(s.search("X"));
19) }
20) }
```

LinkedList:

- It was introduced in JDK1.2 version.
- It is not Legacy Collection.
- It is a direct implementation class to List interface.
- It is index based.
- It allows duplicate elements.
- It follows insertion order.
- It is not following sorting order.
- It allows heterogeneous elements.
- It allows null values in any number.
- Its internal data structure is "Double Linked List".;
- It is best choice for frequent insertions and deletions.
- It is not synchronized Collection.
- No method is synchronized in LinkedList.
- It allows more than one thread to access data.
- It will follow parallel execution.
- It will decrease execution time.
- It will improve application performance.
- It is not giving guarantee for data consistency.
- It is not threadsafe.

Constructors:

- **public LinkedList()**

It will create an empty LinkedList object.

EX: LinkedList ll = new LinkedList();

- **public LinkedList(Collection c)**

It will create LinkedList object with all the elements of the specified Collection object.

EX:

```
LinkedList ll1 = new LinkedList();
```

```
ll1.add("A");
```

```
ll1.add("B");
```

```
ll1.add("C");
```

```
ll1.add("D");
```

```
System.out.println(ll1);  
LinkedList ll2=new LinkedList(ll1);  
System.out.println(ll2);
```

OUTPUT: [A, B, C, D]
[A, B, C, D]

Methods:

- **public void addFirst(Object obj)**
It will add the specified element as first element to LinkedList.
- **public void addLast(Object obj)**
It will add the specified element as last element to LinkedList.
- **public Object getFirst()**
It will return first element from LinkedList.
- **public Object getLast()**
It will return last element from LinkedList.
- **public void removeFirst()**
It will remove first element from LinkedList.
- **public void removeLast()**
It will remove last element from LinkedList.

EX:

```
1) import java.util.*;  
2) class Test  
3) {  
4)     public static void main(String[] args)  
5)     {  
6)         LinkedList ll=new LinkedList();  
7)         ll.add("A");  
8)         ll.add("B");  
9)         ll.add("C");  
10)        ll.add("D");  
11)        ll.add("E");  
12)        System.out.println(ll);  
13)        ll.addFirst("X");  
14)        ll.addLast("Y");
```

```
15)    System.out.println(ll);
16)    ll.removeFirst();
17)    ll.removeLast();
18)    System.out.println(ll);
19)    System.out.println(ll.getFirst());
20)    System.out.println(ll.getLast());
21)    }
22) }
```

Cursors / Iterators in Collections:

In java applications, when we pass Collection object reference variable as parameter to System.out.println(-) method, then, JVM will execute toString() method internally. Initially toString() method was implemented in java.lang.Object class, it was implemented in such a way that to return a String contains "Class_Name@Ref_val" . In java applications, Collection classes are not depending on Object class toString() method, they are having their own toString() method , which are implemented in such a way to return a String contains all the elements of the Collection object by enclosed with [].

EX:

```
ArrayList al = new ArrayList();
al.add("A");
al.add("B");
al.add("C");
al.add("D");
System.out.println(al);
```

OUTPUT: [A, B, C, D]

As per the requirement, we don't want to display all the Elements at a time on command prompt, we want to retrieve elements one by one individually from Collection objects and we want to display all the elements one by one on Command prompt.

To achieve the above requirement, Collection Framework has provided the following three Cursors or Iterators.

- Enumeration
- Iterator
- ListIterator

- **Enumeration:**

- It is a Legacy Cursor, it is applicable for only Legacy Collections to retrieve elements in one by one fashion.
- To retrieve elements from Collections by using Enumeration we have to use the following steps.

- **Create Enumeration Object:**

To create Enumeration object we have to use the following method from Legacy Collections.

`public Enumeration elements()`

- **Retrive Elements from Enumeration:**

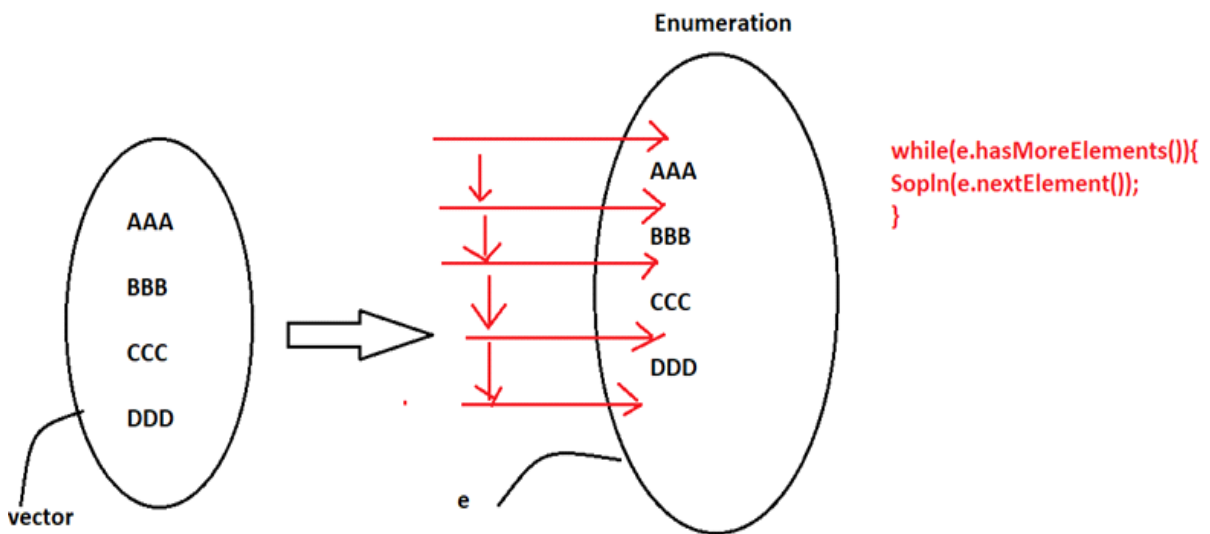
- Check whether more elements are available or not from Current cursor position by using the following method.

`public boolean hasMoreElements()`

- It will return true value if at least next element is existed.
- It will return false value if no element is existed from current cursor position.

- If at least next element is existed then read next element and move cursor to next position by using the following method.

`public Object nextElement()`



EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         Vector v=new Vector();
7)         v.add("A");
8)         v.add("B");
9)         v.add("C");
10)        v.add("D");
11)        v.add("E");
12)        System.out.println(v);
13)        Enumeration e=v.elements();
14)        while(e.hasMoreElements())
15)        {
16)            System.out.println(e.nextElement());
17)        }
18)    }
19)}
```

Drawbacks:

- Enumeration is applicable for only Legacy Collections.
- Enumeration is able to allow only read operation while iterating elements.

• Iterator:

- Iterator is an interface provided JAVA along with its JDK1.2 version.
- Iterator can be used to retrieve all the elements from Collection objects in one by one fashion.
- Iterator is applicable for all the Collection interface implementation classes to retrieve elements.
- Iterator is able to allow both read and remove operations while iterating elements.
- If we want to use Iterator in java applications then we have to use the following steps.

• Create Iterator Object:

- To create Iterator object we have to use the following method from all Collection implementation classes.
- public Iterator iterator()
- EX: Iterator it = al.iterator();

- **Retrive Elements from Iterator:**

- To retrieve elements from Iterator we have to use the following steps.
- Check whether next element is existed or not from the current cursor position by using the following method.

public boolean hasNext()

- This method will return true if next element is existed.
 - This method will return false if no element is existed from current cursor position.
- If next element is existed then read next element and move cursor to next position by using the following method.
 - `public Object next()`

NOTE: To remove an element available at current cursor position then we have to use the following method

`public void remove()`

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         ArrayList al=new ArrayList();
7)         al.add("A");
8)         al.add("B");
9)         al.add("C");
10)        al.add("D");
11)        al.add("E");
12)        System.out.println(al);
13)        Iterator it=al.iterator();
14)        while(it.hasNext())
15)        {
16)            String element=(String)it.next();
17)            System.out.println(element);
18)            if(element.equals("C"))
19)            {
20)                it.remove();
21)            }
22)        }
23)        System.out.println(al);
24)    }
25) }
```

Q) What are the differences between *Enumeration* and *Iterator*?

- Enumeration is Legacy Cursor, it was introduced in JDK1.0 version.
Iterator is not Legacy Cursor, it was introduced in JDK1.2 version.
- Enumeration is not Universal Cursor, it is applicable for only Legacy Collections.
Iterator is an universal Cursor, it is applicable for all Collection implementations.
- Enumeration is able to allow only read operation while iterating elements.
Iterator is able to allow both read operation and remove operation while iterating elements.

• ListIterator:

- It is an interface provided by JAVA along with JDK1.2 version.
- It able to allow to read elements in both forward direction and backward direction.
- It able to allow the operations like read, insert, replace and remove while iterating elements.
- If we want to use ListIterator in java applications then we have to use the following steps.

• Create ListIterator Object:

- To create ListIterator object we have to use the following method.
- `public ListIterator listIterator()`
- EX: `ListIterator lit = ll.listIterator();`

• Retrieve Elements from ListIterator

To retrieve elements from ListIterator in Forward direction then we have to use the following methods.

public boolean hasNext()

It will check whether next element is existed or not from the current cursor position.

public Object next()

It will return next element and it will move cursor to the next position in forward direction.

public int nextIndex()

It will return next index value from the current cursor position.

To retrieve elements in Backward direction we have to use the following methods.

public boolean hasPrevious()

It will check whether previous element is existed or not from the current cursor position, If previous element is existed then it will return "true" value, if previous element is not existed then it will return false value.

public Object previous()

It will return previous element and it will move cursor to the next previous position.

public int previousIndex()

It will return previous index value from the current cursor position.

To perform the operations like remove, insert and replace over the elements we have to use the following methods.

- `public void remove()`
- `public void add(Object obj)`
- `public void set(Object obj)`

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         LinkedList ll=new LinkedList();
7)         ll.add("A");
8)         ll.add("B");
9)         ll.add("C");
10)        ll.add("D");
11)        ll.add("E");
12)        ll.add("F");
13)        System.out.println(ll);
14)        ListIterator lit=ll.listIterator();
15)        System.out.println("Elements in Forward Direction");
16)        while(lit.hasNext())
17)        {
18)            System.out.println(lit.nextIndex()+"--->"+lit.next());
19)        }
20)        System.out.println();
21)        System.out.println("Elements in Backward Direction");
22)        while(lit.hasPrevious())
23)        {
24)            System.out.println(lit.previousIndex()+"--->"+lit.previous());
```

```
25)    }  
26) }  
27)}
```

EX:

```
1) import java.util.*;  
2) class Test1  
3) {  
4)     public static void main(String[] args)  
5)     {  
6)         LinkedList ll=new LinkedList();  
7)         ll.add("A");  
8)         ll.add("B");  
9)         ll.add("C");  
10)        ll.add("D");  
11)        ll.add("E");  
12)        ll.add("F");  
13)        System.out.println(ll);  
14)        ListIterator lit=ll.listIterator();  
15)        while(lit.hasNext())  
16)        {  
17)            String element=(String)lit.next();  
18)            if(element.equals("B"))  
19)            {  
20)                lit.add("X");  
21)            }  
22)            if(element.equals("D"))  
23)            {  
24)                lit.set("Y");  
25)            }  
26)            if(element.equals("E"))  
27)            {  
28)                lit.remove();  
29)            }  
30)        }  
31)        System.out.println(ll);  
32)    }  
33)}
```

Q) What are the differences between *Enumeration*, *Iterator* and *ListIterator*?

- Enumeration is applicable for only Legacy Collections.
Iterator is applicable for all Collection implementations.
ListIterator is applicable for only List implementations.
- Enumeration and Iterator are allowed to iterate elements in only Forward direction.
ListIterator is able to allow to iterate elements in both Forward direction and backward direction.
- Enumeration is able to allow only read operation while iterating elements.
Iterator is able to allow both read and remove operations while iterating elements.
ListIterator is able to allow the operations like insert, replace, remove and read operations while iterating elements.

Set:

- It was introduced in JDK 1.2 version.
- It is a direct child interface to Collection interface.
- It is not index based, it able to arrange all the elements on the basis of elements hashcode values.
- It will not allow duplicate elements.
- It will not follow insertion order.

Note: LinkedHashSet will follow insertion order.

- It will not follow Sorting order.

Note: SortedSet, NavigableSet and TreeSet are following Sorting order.

- It able to allow only one null value.

Note: SortedSet, NavigableSet and TreeSet are not allowing even single null value.

HashSet:

- HashSet is a direct implementation class to Set interface.
- It was introduced in JDK 1.2 version.
- It is not index based, it able to arrange all the elements on the basis of elements hashcode values.
- It will not allow duplicate elements.
- It will not follow insertion order.
- It will not follow Sorting order.
- It able to allow only one null value.
- Its internal data structure is "Hashtable".
- Its initial capacity is "16" elements and its initial fill_Ratio is 75%.
- It is not synchronized.
- Almost all the methods are not synchronized in HashSet

- It allows more than one thread at a time.
- It follows parallel execution.
- It will reduce execution time.
- It improves performance of the applications.
- It is not giving guarantee for data consistency.
- It is not threadsafe.

Constructors:

- **public HashSet()**

This constructor can be used to create an empty HashSet object with 16 elements as initial capacity and 75% fill ratio.

EX: `HashSet hs = new HashSet();`

- **public HashSet(int capacity)**

This constructor can be used to create an empty HashSet object with the specified capacity as initial capacity and with the default fill ratio 75%.

EX: `HashSet hs = new HashSet(20);`

- **public HashSet(int capacity, float fill Ratio)**

This constructor can be used to create an empty HashSet object with the specified capacity and with the specified fill ratio.

EX: `HashSet hs = new HashSet(20, 0.85f);`

- **public HashSet(Collection c)**

This constructor can be used to create HashSet object with all the elements of the specified Collection.

EX:

```

1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         HashSet hs1=new HashSet();
7)         hs1.add("A");
8)         hs1.add("B");
9)         hs1.add("C");
10)        hs1.add("D");
11)        hs1.add("E");
12)        System.out.println(hs1);
13)        HashSet hs2=new HashSet(hs1);

```

```
14) System.out.println(hs2);  
15) }  
16) }
```

OUTPUT:

[D,E,A,B,C]

[D,E,A,B,C]

EX:

```
1) import java.util.*;  
2) class Test  
3) {  
4)     public static void main(String[] args)  
5)     {  
6)         HashSet hs=new HashSet();  
7)         hs.add("A");  
8)         hs.add("B");  
9)         hs.add("C");  
10)        hs.add("D");  
11)        hs.add("E");  
12)        System.out.println(hs);  
13)        hs.add("B");  
14)        System.out.println(hs);  
15)        hs.add(null);  
16)        hs.add(null);  
17)        System.out.println(hs);  
18)        hs.add(new Integer(10));  
19)        System.out.println(hs);  
20)    }  
21) }
```

LinkedHashSet:

Q) What are the differences between *HashSet* and *LinkedHashSet*?

- HashSet was introduced in JDK 1.2 version.
LinkedHashSet was introduced in JDK 1.4 version.
- HashSet is not following insertion order.
LinkedHashSet is following insertion order.
- The internal data structure of HashSet is "Hashtable".
The internal data structure of LinkedHashSet is "Hashtable" and "LinkedList".

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         HashSet hs=new HashSet();
7)         hs.add("A");
8)         hs.add("B");
9)         hs.add("C");
10)        hs.add("D");
11)        hs.add("E");
12)        System.out.println(hs);
13)        LinkedHashSet lhs=new LinkedHashSet();
14)        lhs.add("A");
15)        lhs.add("B");
16)        lhs.add("C");
17)        lhs.add("D");
18)        lhs.add("E");
19)        System.out.println(lhs);
20)    }
21)}
```

OUTPUT:

[D,E,A,B,C]

[A,B,C,D,E]

SortedSet:

- It was introduced in JDK1.2 version.
- It is a child interface to Set interface.
- It is not index based.
- It is not allowing duplicate elements.
- It is not following insertion order.
- It follows Sorting order.
- It allows only homogeneous elements.
- It will not allow heterogeneous elements, if we are trying to add heterogeneous elements then JVM will rise an exception like java.lang.ClassCastException.
- It will not allow null values, if we are trying to add any null value then JVM will rise an exception like java.lang.NullPointerException.
- It able to allow only Comparable objects by default, if we are trying to add non comparable objects then JVM will rise an exception like java.lang.ClassCastException.

Note: If we are trying to add non comparable objects then we have to use Comparator.

Methods:

- **public Object first()**

It will return first element from SortedSet.

- **public Object last()**

It will return last element from SortedSet.

- **public SortedSet headSet(Object obj)**

It will return SortedSet object with the elements which are less the specified element.

- **public SortedSet tailSet(Object obj)**

It will return SortedSet object with the elements which are greater than or equals to the specified element.

- **public SortedSet subSet(Object obj1, Object obj2)**

It will return SortedSet object with all elements which are greater than or equals to the specified first element and which are less than the specified second element.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         TreeSet ts=new TreeSet();
7)         ts.add("D");
8)         ts.add("F");
9)         ts.add("B");
10)        ts.add("E");
11)        ts.add("C");
12)        ts.add("A");
13)        System.out.println(ts);
14)        System.out.println(ts.first());
15)        System.out.println(ts.last());
16)        System.out.println(ts.headSet("D"));
17)        System.out.println(ts.tailSet("D"));
18)        System.out.println(ts.subSet("B","E"));
19)    }
20) }
```

NavigableSet

It was introduced in JAVA6 version, it is a child interface to SortedSet interface, it is following all the properties of SortedSet and it has define methods to provide navigations over the elements.

Methods:

- **public Object ceiling(Object obj)**

It will return lowest element among all the elements which are greater than or equals to the specified element.

- **public Object higher(Object obj)**

It will return lowest element among all the elements which are greater than the specified element.

- **public Object floor(Object obj)**

It will return highest element among all the elements which are less than or equals to the specified element.

- **Trpublic Object lower(Object obj)**

It will return highest element among all the elements which are less than the specified element.

- **public Object pollFirst()**

It will remove and return first element from NavigableSet.

- **public Object pollLast()**

It will remove and return last element from NavigableSet.

- **public NavigableSet descendingSet()**

It will return all elements in the form of NavigableSet in descending order.

EX:

```
1) import java.util.*;
2) class Test
3) {
4)     public static void main(String[] args)
5)     {
6)         TreeSet ts=new TreeSet();
7)         ts.add("D");
```

```

8)    ts.add("F");
9)    ts.add("B");
10)   ts.add("E");
11)   ts.add("C");
12)   ts.add("A");
13)   System.out.println(ts);
14)   System.out.println(ts.ceiling("D"));
15)   System.out.println(ts.higher("D"));
16)   System.out.println(ts.floor("D"));
17)   System.out.println(ts.lower("D"));
18)   System.out.println(ts.descendingSet());
19)   ts.pollFirst();
20)   ts.pollLast();
21)   System.out.println(ts);
22) }
23) }

```

TreeSet:

- It was introduced in JDK 1.2 version.
- It is not Legacy Collection.
- It has provided implementation for Collection, Set, SortedSet and NavigableSet interfaces.
- It is not index based.
- It is not allowing duplicate elements.
- It is not following insertion order.
- It follows Sorting order.
- It allows only homogeneous elements.
- It will not allow heterogeneous elements, if we are trying to add heterogeneous elements then JVM will rise an exception like `java.lang.ClassCastException`.
- It will not allow null values, if we are trying to add any null value then JVM will rise an exception like `java.lang.NullPointerException`.
- It able to allow only Comparable objects by default, if we are trying to add non comparable objects then JVM will rise an exception like `java.lang.ClassCastException`.

NOTE: If we are trying to add non comparable objects then we have to use `java.util.Comparator`.

- Its internal data structure is "Balanced Tree".
- It is mainly for frequent search operations.